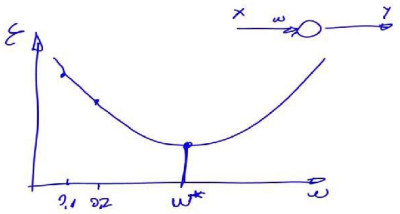
UNIDAD 4 - Aprendizaje Automático

Capacidad de generalización

Superficies de error

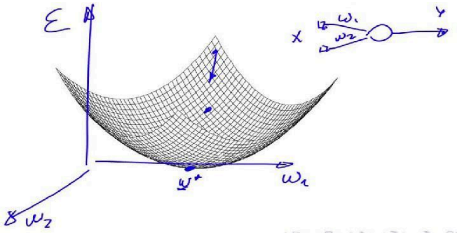
Suponiendo que hay un único parámetro (peso w) a ajustar:

- Podemos medir un error que comete el sistema con distintos valores del parámetro real w.
- => podríamos graficar una curva de error

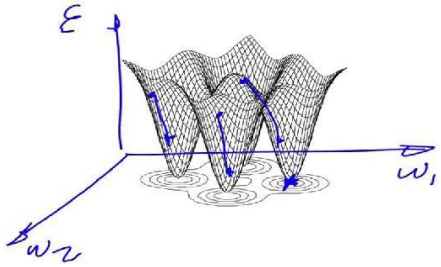


Con 2 dimensiones:

- Podemos graficar una superficie de error:



¿Y si hay muchos mínimos?



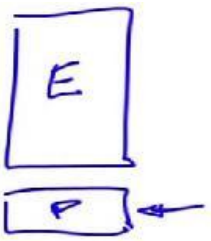
→ dependiendo de dónde arranque el algoritmo de entrenamiento, va a elegir uno u otro, sin encontrar el **mínimo global**

Una opción → inicializar el alg. con varios puntos al azar y quedarnos con el mínimo.

Estos errores los medimos durante el entrenamiento, pero...

- ¿El modelo es *capaz* de funcionar bien en casos más *generales*?
- ¿Cuál será el error en casos nunca vistos durante el entrenamiento?

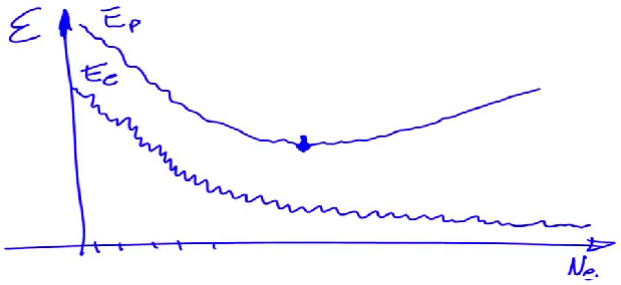
=> para ello hacemos una prueba.



SOBRE-ENTRENAMIENTO:

→ Si un modelo se ajusta demasiado a los datos de entrenamiento y pierde la capacidad de funcionar bien con datos que nunca vió.

- Si observamos ***la cantidad de épocas*** **Ne** contra los errores:

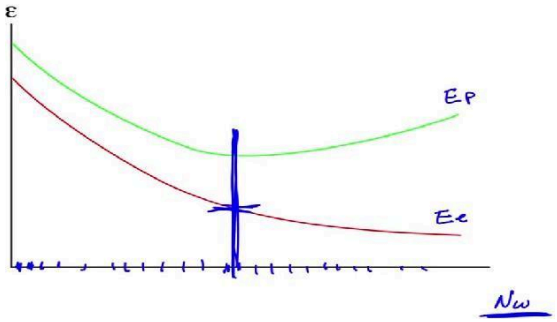


N_e → número de épocas.
E_e → error entrenamiento.
E_p → error prueba.

Vemos que luego de un cierto número de épocas, *el error de la prueba se va a estancar, y luego va a empezar a subir.*

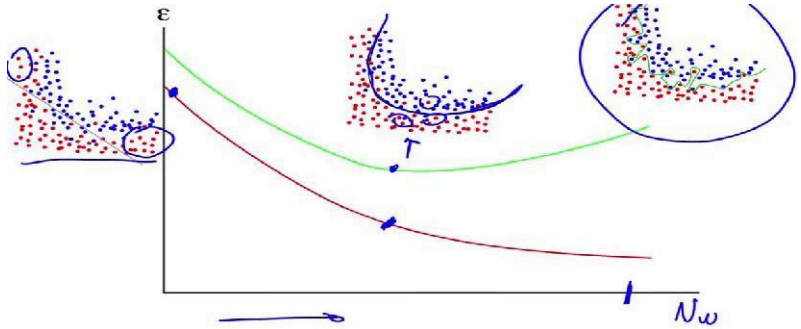
→ Esto se debe a que **el modelo se aprendió detalles de los datos de entrenamiento que no son detalles generales del problema que debe aprender realmente**.

- Si en lugar de la cantidad de épocas, *ahora usamos la* ***cantidad de neuronas*** **Nw**, parámetros libres, que tiene la red:



→ Observamos un comportamiento similar al anterior.

→ Ya que a mayor cantidad de parámetros libres, más capacidad tiene la red para modelar todos los detalles finos que tienen los datos de entrenamiento.



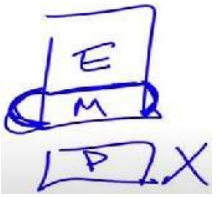
¿Qué puedo hacer para EVITAR EL SOBRE-ENTRENAMIENTO?

→ *Cuando vemos que el error de prueba sube, cortamos el entrenamiento*.

→¿Pero no estamos ajustando según los datos de **prueba** entonces?
→ sí, y en realidad estos no deberán usarse para nada

=> tenemos que tener una estrategia para medir sólo con los de entrenamiento:

easy → nos guardamos una porción de los datos de entrenamiento y le llamamos un subconjunto de monitoreo.



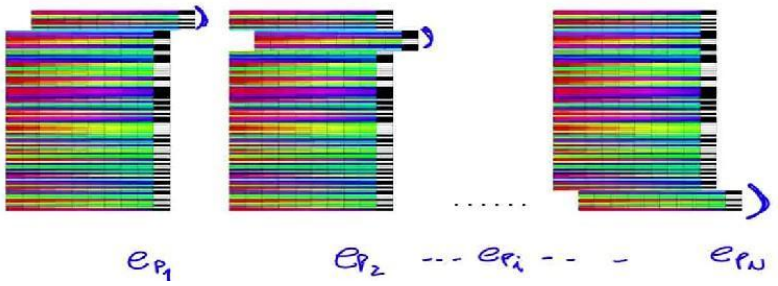
¿Cómo podemos ESTIMAR la capacidad de generalización?

Validación cruzada

a. Leave-k-out

Rep gráfica: color según las intensidades de los valores

1. Reordenar al azar los datos.
2. Dejo k casos afuera para la prueba, el resto para entrenar, y podemos tener un subset de monitoreo en él para ir midiendo durante el entrenamiento.
3. Hago el entrenamiento.
4. Vuelvo a comenzar de cero y repito este mecanismo tomando otro conjunto de prueba para variar los datos que estaban en cada uno de los 3 conjuntos.
5. Vamos a terminar con N errores de prueba.



6. Finalmente calculamos el **error promedio** como el promedio de ellos:

$$\mathcal{E} = \frac{1}{N} \sum_{i=1}^{N} e_{P_i}$$

y **este va a ser el verdadero error que me interesa**.

→ También es útil medir la **VARIANZA del error**:

- Si es pequeña => las particiones son buenas.
- Si es grande => es porque hay valores de errores muy diferentes entre ellos, lo cual nos indicaría que hay algún problema con los datos.

b. Leave-1-out

- Consiste en entrenar y luego probar con 1 solo ejemplo, lo cual implica, si tenemos N datos, repetirlo N veces.
- El problema es que es pesado.
- Pero como ventaja, tiene un sesgo del error que estima muy pequeño.

c. Solapar las particiones

- Pierdo la independencia estadística de los experimentos

Medidas de desempeño para clasificadores (binarios)

Matriz de confusión

		Predicciones ⊕	Predicciones ⊖	
Clases correctas	⊕	$t_\oplus$	$f_\ominus$	$N_\oplus = t_\oplus + f_\ominus$
	⊖	$f_\oplus$	$t_\ominus$	$N_\ominus = f_\oplus + t_\ominus$
				$N = N_\oplus + N_\ominus$

- Verdaderos positivos → dijo + y era +
- Falsos negativos → dijo - y era +

→ sumados = cantidad de positivos

- Falsos positivos → dijo + y era -
- Verdaderos negativos → dijo - y era -

→ sumados = cantidad de positivos

→ sumando ambas cantidades = total de casos

- **Sensibilidad**

$$s^+ = \frac{t_\oplus}{t_\oplus + f_\ominus} = \frac{t_\oplus}{N_\oplus} \longrightarrow \left\{ \begin{array}{l} \textbf{sensitivity} \\ \text{true positive rate} \\ \text{hit rate} \\ \text{recall} \\ \text{power} \end{array} \right.$$

- **Especificidad**

$$s^- = \frac{t_\ominus}{t_\ominus + f_\oplus} = \frac{t_\ominus}{N_\ominus} \longrightarrow \left\{ \begin{array}{l} \textbf{specificity} \\ \text{true negative rate} \end{array} \right.$$

- **Precisión**

$$p = \frac{t_\oplus}{t_\oplus + f_\oplus} \longrightarrow \left\{ \begin{array}{l} \textbf{precision} \\ \text{positive predictive value} \end{array} \right.$$

$$n = \frac{t_\ominus}{t_\ominus + f_\ominus} \longrightarrow \left\{ \text{negative predictive value} \right.$$

Exactitud y otras medidas compuestas

- **Exactitud**
  Es la más intuitiva y la más utilizada, se suman los verdaderos positivos y los verdaderos negativos y luego lo dividimos por la cantidad de casos.

$$a = \frac{t_\oplus + t_\ominus}{N} \longrightarrow \left\{ \textbf{accuracy} \right.$$

- **F1 score**
  Está focalizada en los positivos, ya que suele ser la que más interesa. Se combina con una media armónica la **sensibilidad** con la **precisión**.

$$F_1 = \frac{2t_{\oplus}}{2t_{\oplus} + f_{\oplus} + f_{\ominus}} = 2\frac{s^+ p}{s^+ + p} \longrightarrow \begin{cases} F_1 \textbf{ score} \\ \text{F-score/measure} \\ \text{harmonic mean} \end{cases}$$

- **Media geométrica**
  También relaciona la sensibilidad y precisión, que también nos da una idea de que también está andando el clasificador.

$$G = \sqrt{s^+ p} \longrightarrow \begin{cases} \textbf{G-measure} \\ \text{geometric mean} \end{cases}$$

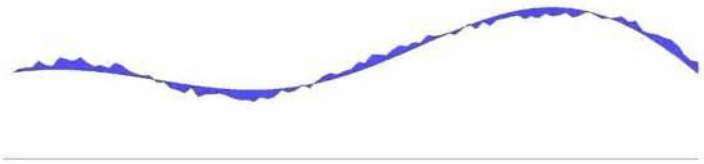
Errores relativos

Tenemos 2 clasificadores, y queremos comparar el desempeño $\mathbf{a_1}$ de uno relativo al $\mathbf{a_2}$ otro:

- $\mathbf{e_r = 1 - a_2}$, $\mathbf{e = 1 - a_1}$
- => el error relativo a la referencia $\mathbf{e_r}$ es: $\delta_e = \frac{e_r - e}{e_r}$

Medidas de desempeño para la predicción en series

La idea es medir con el área entre la referencia y la curva que se predijo:



- Error medio absoluto → $\tilde{\varepsilon}_A = \frac{1}{N}\sum_{i=1}^{N} |e_i|$ (MAE/MAD)
- Error cuadrático medio → $\tilde{\varepsilon}_S = \frac{1}{N}\sum_{i=1}^{N} e_i^2$ (MSE)
- Raíz cuadrada del error cuadrático medio → $\tilde{\varepsilon}_R = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}$ (RMSE)

Aprendizaje automático “no neuronal”

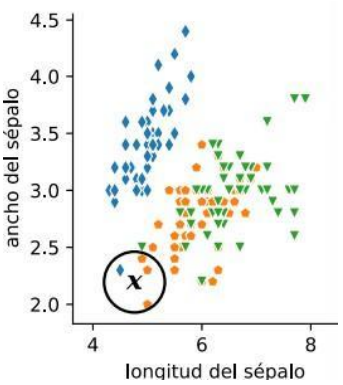
kNN - k-Nearest Neighbors

- Método NO paramétrico
- En realidad no aprende
- Vuelve a buscar los más cercanos con cada nuevo caso a clasificar (basado en instancias).

Clasificador kNN

Supongamos que tenemos:

- **N_k** puntos de la clase **C_k**
- **N** puntos en total $(\sum_k N_k = N)$



Para un nuevo **x**, podemos imaginar una esfera centrada en **x**,

→ Supongamos que esta esfera:

- tiene un volumen **V**
- contiene **K** puntos en total
- de los cuales **Kk** son de la clase **Ck**

La ***densidad de la clase k*** en la esfera sería → $\mathrm{p}(\mathbf{x}|C_k) = \frac{K_k}{N_k V}$

Y *si no condicionamos a la clase* tendríamos → $\mathrm{p}(\mathbf{x}) = \frac{K}{NV}$

Lo que podríamos esperar para cada clase sería → $\mathrm{p}(C_k) = \frac{N_k}{N}$

→ Si con esto usamos el **teorema de Bayes**

$$\mathrm{p}(C_k|\mathbf{x}) = \frac{\mathrm{p}(\mathbf{x}|C_k)\mathrm{p}(C_k)}{\mathrm{p}(\mathbf{x})} = \frac{\frac{K_k}{N_k V}\frac{N_k}{N}}{\frac{K}{NV}} = \frac{K_k}{K}$$

= probabilidad de que el patrón dado que sea de la clase, la probabilidad del patrón / probabilidad de clase

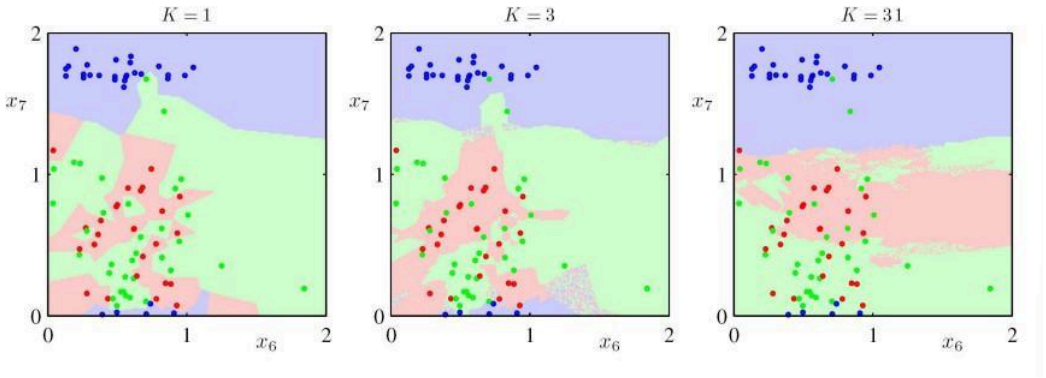
Ventajas:

- ✔ Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- ✔ No hay parámetros entrenables
- ✔ Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento
- ✔ Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

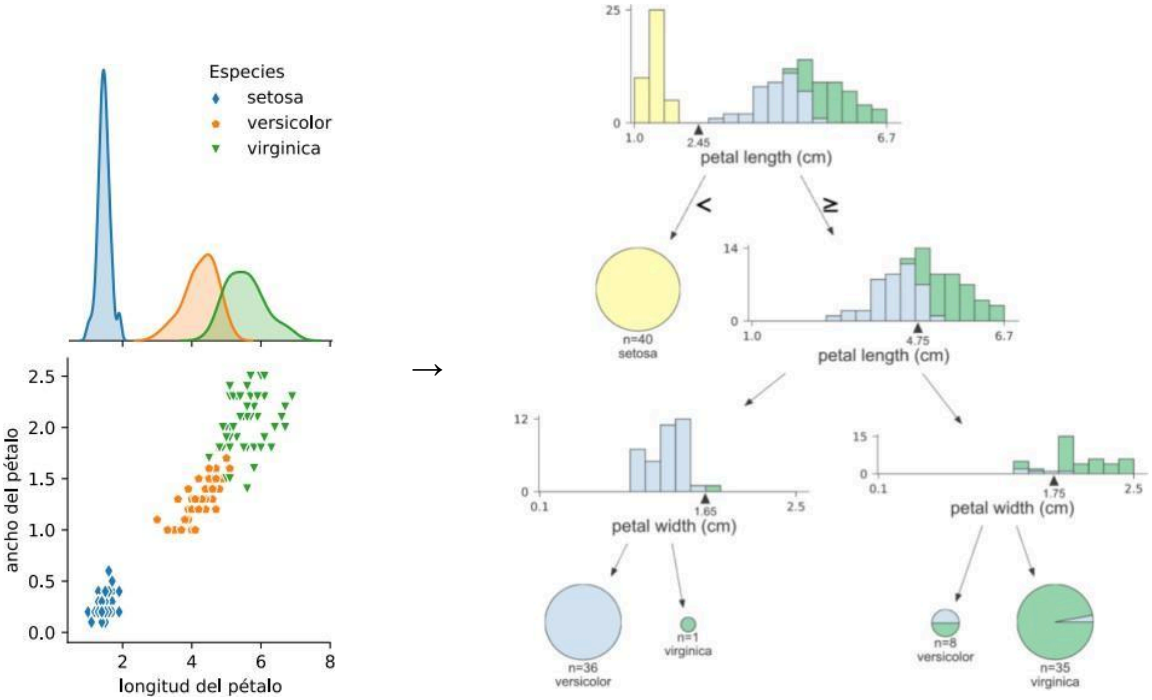
- ✖ Hay que determinar el valor óptimo de k.
- ✖ No es tan claro que distancia usar en cada caso.
- ✖ Para cada dato de prueba se requiere calcular la distancia a todos los datos de entrenamiento.
- ✖ Sensible a outliers y clases desbalanceadas.

A más vecinos, más detalle:



(Voronoi)

Árboles de decisión



- ¿Cómo elegimos por qué variable dividir primero?
- ¿Cómo elegimos el punto de división?

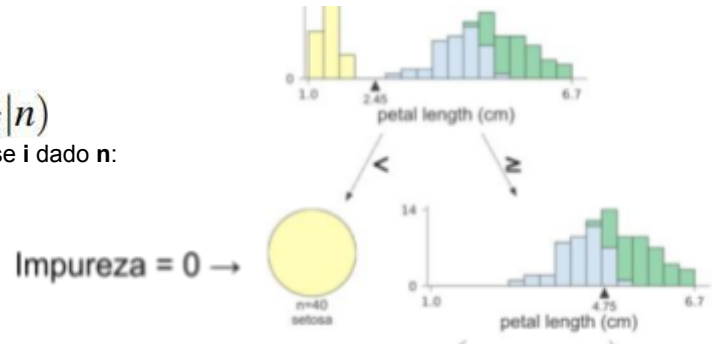
→ Se usan los índices de impurezas:

Medidas de impureza

Impureza de clasificación:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Probabilidad de que **x** pertenezca a la clase **i** dado **n**:



→ Empezamos con la variable que nos de menos impureza, según el n del cálculo.

Impureza de Gini: → Es la que más se usa

$$\mathcal{I}_n^G = \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i|n)\mathsf{P}(\mathbf{x} \in w_j|n)$$
$$= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i|n)$$

= sum(Probabilidad de que **x** pertenezca a la clase **i** dado **n***Probabilidad de que **x** pertenezca a la clase **j** dado **n)**

Impureza de Entropía: *(Entropía → "Desorden")*

$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i|n) \log \mathsf{P}(\mathbf{x} \in w_i|n)$$

→ Si queda todo de un lado, w_i, nos va dar cero (son todos iguales).

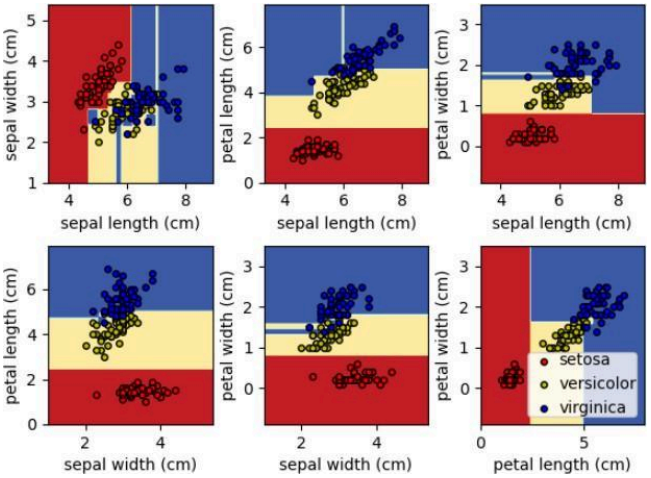
Ganancia de información:

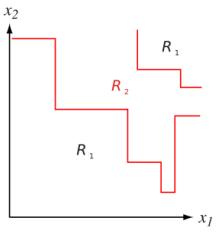
G = $\mathcal{I}$ **p** $-\sum$ **(Nh/N** $\mathcal{I}$ **h)**

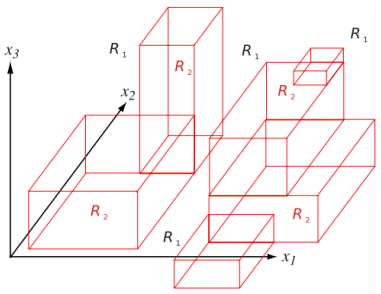
= impureza del padre - impureza de sus hijos que quedan

→ Mide cuánta información gané haciendo esa partición de la información.

Fronteras de decisión:







Ventajas:

- ✔ Proveen una ***explicación simple de la decisión*** con reglas IF-THEN.
- ✔ Trata de forma natural tanto *datos numéricos como categóricos* (y a la vez) sin requerir transformaciones.
- ✔ Tienen *bajo costo computacional* tanto para el entrenamiento como para su uso en producción.

✔ No requieren grandes cantidades de datos para entrenar.
✔ Bastante *robustos a los outliers*.

Desventajas:

✖ *No son robustos a ruido* (pequeños cambios) en los datos.
✖ *Fronteras de decisión ortogonales a las dimensiones* de entrada.
✖ No escalan bien con la complejidad de los problemas a resolver.
✖ *Se sobreajustan* con facilidad.

Clasificadores Bayesianos Ingenuos (Naive Bayes)

Construyamos un clasificador usando el teorema de Bayes:

$$\mathsf{P}(w|\mathbf{x}) = \frac{\mathsf{P}(\mathbf{x}|w)\mathsf{P}(w)}{\mathsf{P}(\mathbf{x})}$$

Probabilidad de que sea la clase w, dado el patrón x

...y supongamos que *las dimensiones o características son condicionalmente independientes* (esta es la parte **más ingenua**):

$$\mathsf{P}(x_i|w, x_j) = \mathsf{P}(x_i|w) \quad \forall i \neq j.$$

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

y usando en cadena esta misma definición:

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, \ldots, x_n) &= \mathsf{P}(x_1, x_2, \ldots, x_n|w)\mathsf{P}(w) \\
&= \mathsf{P}(x_2, x_3, \ldots, x_n|w, x_1)\mathsf{P}(w)\mathsf{P}(x_1|w) \\
&= \mathsf{P}(x_3, x_4, \ldots, x_n|w, x_1, x_2)\mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w, x_1) \\
&= \ldots
\end{aligned}$$

pero con la suposición ingenua:

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, .., x_n) &= \mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w)\ldots\mathsf{P}(x_n|w) \\
&= \mathsf{P}(w)\prod_{i=1}^{n}\mathsf{P}(x_i|w)
\end{aligned}$$

Me quedan las probabilidades de cada columna (entrada), dada la clase, las cuales podemos calcular fácil con los datos.

De esta forma nos queda

$$\mathsf{P}(w|\mathbf{x}) = \frac{\mathsf{P}(\mathbf{x}|w)\mathsf{P}(w)}{\mathsf{P}(\mathbf{x})} = \frac{\mathsf{P}(w)\prod_{i=1}^{n}\mathsf{P}(x_i|w)}{\mathsf{P}(\mathbf{x})}$$

Y considerando la misma probabilidad P(w) para cada clase, y que P(x) no depende de w, la clase más probable se puede obtener mediante:

$$\hat{w}(\mathbf{x}) = \arg\max_{w} \prod_{i=1}^{n} \mathrm{P}(x_i|w)$$

Para cada clase medís la productoria para cada clase,, y te quedas con la que da más grande.

NB: solo nos queda calcular las P(x_i|w)

NB Gaussiano $\mathcal{N}(\mu, \sigma^2)$

$$\mathrm{P}(x_i|w) = \frac{1}{\sigma_w\sqrt{2\pi}} e^{-\frac{(x_i-\mu_w)^2}{2\sigma_w^2}}$$ (caso continuo)

Ventajas:

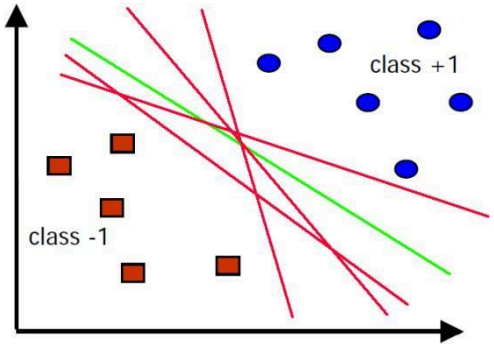
- ✔ Puede usarse directamente en mezclas de datos continuos y categóricos.
- ✔ En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- ✔ El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- ✔ Rápido y simple de implementar.

Desventajas:

- ✖ La hipótesis de independencia no se cumple en muchos datos reales.
- ✖ Suele presentar problemas de estimación durante el entrenamiento.

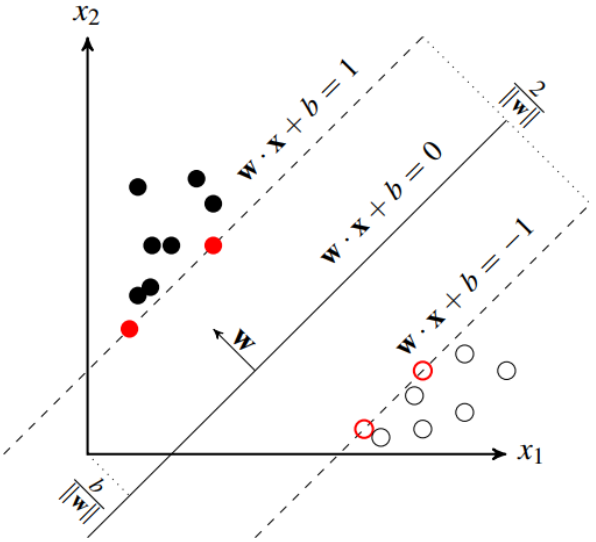
Máquinas con vectores de soporte - (SVM Support Vector Machines)

<u>Margen: ¿cuál sería el mejor clasificador?</u>

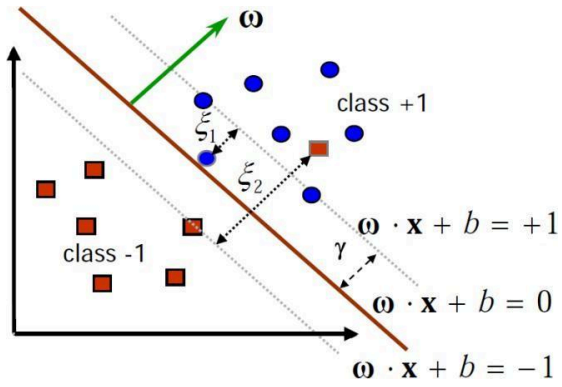


→ recta lo más lejos posible de los patrones con los que empieza la clase

→ Se maximiza el márgen:



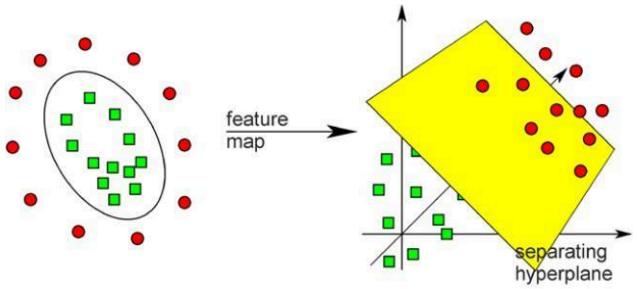
SVM: márgenes blandos



→ Medidas respecto de los nodos que elegimos de soporte.

El truco del kernel

Proyectar los datos a una dimensión superior para que yo pueda atravesar con un hiperplano a los datos.



$$\phi : \mathbb{R}^2 \to \mathbb{R}^3$$

$$\phi : (x_1, x_2) \mapsto (z_1, z_2, z3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$
- Polinomial: $(\mathbf{x} \cdot \mathbf{z} + c)^d$
- Sigmoideo: $\tanh\left(c(\mathbf{x} \cdot \mathbf{z}) + \theta\right)$

Ventajas:

- ✔ *Rápida convergencia* en entrenamiento.
- ✔ Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- ✔ Funciona bastante *bien en altas dimensiones de entrada*.
- ✔ Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- ✔ Suele proveer buena capacidad de generalización. (maximizan el márgen)

Desventajas:

- ✖ El *desempeño es muy dependiente del kernel* que se elija (y sus hiperparametros).
- ✖ Tiene problemas con grandes conjuntos de datos.
- ✖ No maneja bien el solapamiento de clases.
- ✖ No provee buena interpretabilidad de las decisiones.
- ✖ No es directa la extensión a problemas multiclase.

Ensambles de clasificadores

→ *Combinar varios modelos simples en lugar de construir un modelo complejo.*

Bagging - Bootstrap AGGregatING

Entrenar varios modelos simples…
haciendo *bootstrap* sobre el conjunto de datos

1. Entrenar ***B*** modelos separados, cada uno para predecir $y_b(x)$.
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(x)$ (o promedio en caso de regresión).

$$yens(x) = \sum_{b=1}^{B} yb(x)$$

Se puede demostrar que el error del ensamble E_E en relación al promedio de los errores individuales $\tilde{E}_b$ es:

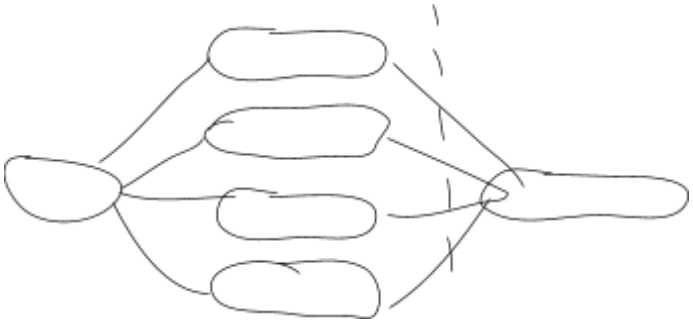
$$\tilde{E}_E = \frac{1}{B}\tilde{E}_b$$

→ Disminuye con la cantidad

(Random Forests)

Stacking

Entrenas varios modelos simples y luego un meta modelo recibe como entrada las salidas de los modelos más simples.



Se entrenan por separado los modelos simples del metamodelo.

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap.
- *Buscamos que la nueva muestra sea más informativa para el nuevo modelo a entrenar.*

Boosting por muestreo 3

Dado un conjunto de entrenamiento ***S*** de ***n*** patrones, para un clasificador **binario**:

1. Se entrena el clasificador ***C1*** usando la muestra de bootstrap **S1** de ***n' < n*** patrones (sin reposición).
2. Se entrena el clasificador ***C2*** en un conjunto de entrenamiento ***S2*** *en el que la mitad de sus patrones hayan sido clasificados correctamente por* ***C1.***
3. Se entrena el clasificador ***C3*** con los patrones en los que la predicción de ***C1*** y ***C2*** difieren.

AdaBoost - ADAptive BOOSTing

La idea clave: *prestar más atención a los patrones difíciles.*

- *Cada patrón de entrenamiento tiene asociado un peso* que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es mal clasificado.
- En test se usa un voto pesado de acuerdo al desempeño previo de cada clasificador del ensamble.

Algoritmo de AdaBoost:

Entrenamiento:

Inicialización: $D_1(i) = 1/n, \quad \forall i.$

para $b = 1, 2, \ldots, B$

1 Muestrear por bootstrap el subconjunto S_b^* de acuerdo a la distribución D_b.

2 Entrenar un clasificador débil con S_b^*, y obtener sus predicciones $h_b : X \to \Omega$

3 Calcular el error de las predicciones h_b:

$$\epsilon_b = \sum_i I(h_b(\mathbf{x}_i), y_i) D_b(i).$$

4 **si** $\epsilon_b \leq \frac{1}{2}$ ← Tiene que ser **menor al "by chance"!**

4.1 Calcular el error normalizado $\beta_b = \frac{\epsilon_b}{1-\epsilon_b}$

4.2 Actualizar la distribución

$$D_{b+1}(i) = \frac{D_b(i)}{Z_b} \times \begin{cases} \beta_b, & \textbf{if } h_b(\mathbf{x}_i) = y_i \\ 1, & \text{otherwise} \end{cases}$$

(Z_b: constante de normalización para las probabilidades).

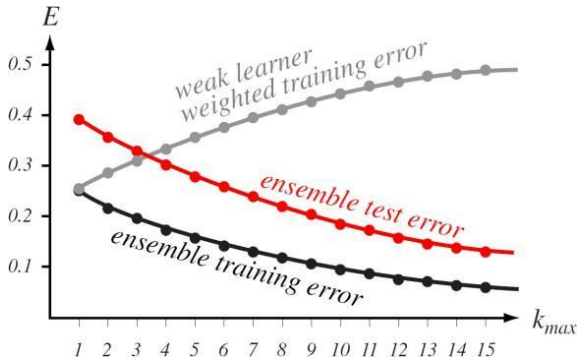
Prueba: Voto por mayoría pesado.

Dado un patrón no etiquetado $\mathbf{z}$

1 Obtener el total de votos recibido por cada clase

$$V_j = \sum_{b:h_b(\mathbf{z})=y_j} \log\left(1/\beta_b\right), \quad \forall j.$$

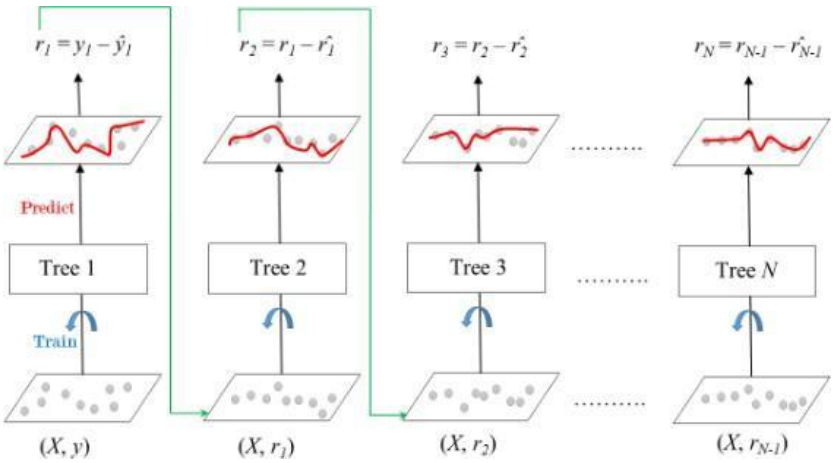
2 Elegir la clase que recibió más votos.



→ aumento cantidad de miembros en el ensamble.

Gradient boosting

- Se usa mucho en **problemas de ML en datos tabulares**
- Es la evolución de los ensambles de los últimos años



En el primer ensamble se hace un pred y a partir de ella se calc un error, que es lo que entra al segundo

→ al segundo ya NO ENTRAN LOS DATOS, lo que estoy tratando de predecir es él es el residuo/error.

Ensambles de clasificadores

Ventajas:

- ✖ *Poco propensos al sobre-entrenamiento* (buena capacidad de generalización).
- ✖ Se adaptan al desbalance de clases.
- ✖ Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
    - Cada tipo de clasificador aprende distintas particularidades del problema.
- ✖ Hacen un mejor aprovechamiento de los datos cuando son escasos.
- ✖ Tratan naturalmente los problemas multiclase.

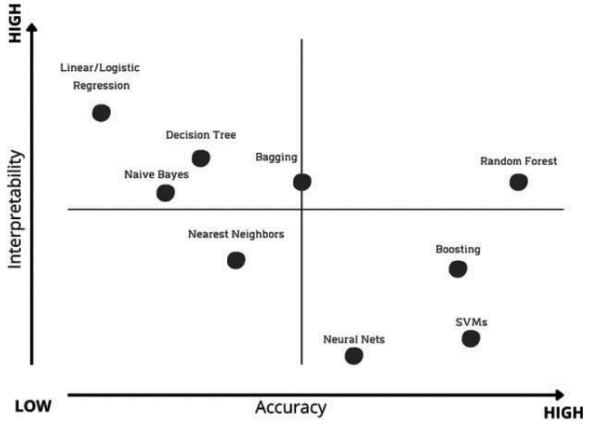
✖ Estables y robustos.

Desventajas:

✖ El entrenamiento puede ser costoso.
✖ No son tan fáciles de interpretar.
✖ Los resultados pueden variar mucho dependiendo de la elección de los modelos débiles, su diversidad y la de los datos.

Interpretabilidad vs exactitud

→ En muchas aplicaciones prácticas, el usuario final del modelo necesita saber el porqué de las salidas.



SOM Mapas auto-organizativos

Auto-organización

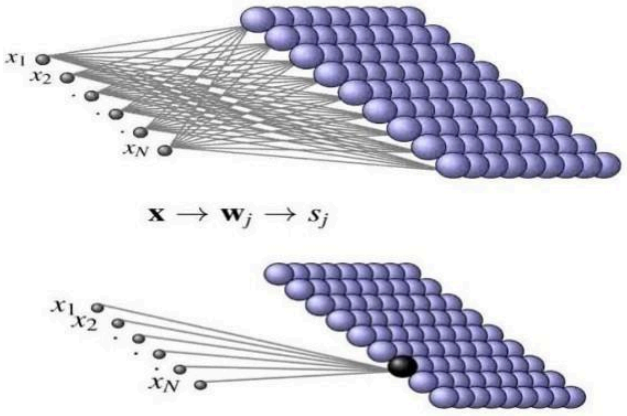
Es el proceso en el cual, *por medio de interacciones locales, se obtiene ordenamiento global.*

- Existen estructuras que se auto-organizan.
- Proceso por el cual los elementos que componen la estructura tienen un comportamiento sencillo y reglas elementales pero emerge un comportamiento global complejo.

Ejemplo: las células del cerebro se auto-organizan en grupos de acuerdo a la información que reciben…

SOM: Arquitectura

- Hay *una sola capa*, y esta organizada en un *plano*.
- Por cada entrada va a haber un peso que conecta a esa entrada con todas las neuronas de la capa, conexión todos contra todos.
    - Tenemos un vector de entradas, una matriz de pesos, y una capa de neuronas.
- Es la más básica y hay muchas variantes.

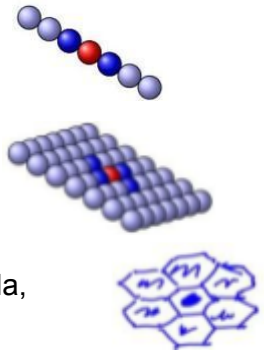


→ Cuando entra un estímulo HAY UNA DE LAS NEURONAS QUE SE ACTIVA, es una **red neuronal de COMPETENCIA**, es decir, se hace una comparación entre la entrada y todos los vectores de pesos, y aquel que tenga menor distancia, o sea el que se parezca más a la entrada, define cuál es la neurona ganadora:

$$j^*(n) = G(\mathbf{x}(n)) = \arg\min_j \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Entornos de influencia o vecindades

➢ Formas básicas de organización:



- **Línea** de neuronas, que cuando una se activa puede afectar a su entorno.
- **Cuadrícula o plano**, que también cuando se activa afecta al resto.
- **Hexagonal** la división del plano que es bastante utilizada, la neurona ganadora tiene 6 neuronas en su vecindad.

➢ Alcance $\mathbf{A_g}$:
A lo largo del entrenamiento estos entornos pueden ser fijos o variables, de radio N.

➢ Excitación asociada al entorno:
→ Las neuronas del entorno pueden estar relacionadas de diferentes formas con la ganadora.
- Excitatorias puras
- Inhibitorias puras
- *Funciones generales* de excitación/inhibición

Funciones de excitación/inhibición lateral

- Uniforme (entorno simple):

$$\Lambda_G(n) = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & \text{si } |G - i| \leq 2 \\ h_{G,i} = 0 & \text{si } |G - i| > 2 \end{cases}$$

→ para un entorno de radio 2, si la distancia entre la neurona ganadora G y una neurona cualquiera del mapa i, es menor que 2, entonces se va a excitar o inhibir con la constante B.
Si la distancia es mayor, no le hace nada

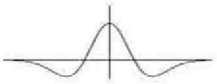
siendo $\beta(n)$ adaptable en función de las iteraciones n.

- Gaussiana:

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

→ si excita o inhibe va a depender del valor de la gaussiana, cuanto más lejos esté, menos la va a influenciar

- Sombrero Mejicano: campos receptivos retina, inhibición lateral



Entrenamiento de un SOM

- Entrenamiento totalmente ***NO-supervisado***.
- **Aprendizaje competitivo**, *en cada iteración no se ajustan los pesos de toda la red neuronal, solo en principio la de la ganadora y algún entorno de ella*.

1. Inicialización:
   - pequeños valores aleatorios con $w_{ji} \in [-0{,}5, \cdots, +0{,}5]$ o también
   - eligiendo aleatoriamente $\mathbf{w}_j(0) = \mathbf{x}_\ell$ $(\ell \in [1, \cdots, L])$
2. Selección del ganador:

$$G(\mathbf{x}(n)) = \arg\min_{\forall j} \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

3. Adaptación de los pesos:

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n)\,(\mathbf{x}(n) - \mathbf{w}_j(n)) & \text{si } y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & \text{si } y_j \notin \Lambda_G(n) \end{cases}$$

→ acercar un poco el valor de los pesos a los valores que trajo el patrón de entrada, según la velocidad de aprendizaje*(diferencia entre entrada y pesos, vector de error), si la neurona y se encuentra en el entorno

4. Volver a 2 hasta no observar cambios significativos.

- <u>Consideraciones prácticas:</u>
  - Generalmente usamos mapas cuadrados.
  - Se suele usar la función $h_{G,i}(n)$ uniforme i, pero con un entorno variable decreciente con **n**.
  - La velocidad de aprendizaje $0 < \eta(n) < 1$ también es variable, decreciente con **n**.

¿Cómo varían $\Lambda_G(n)$ y $\eta(n)$?

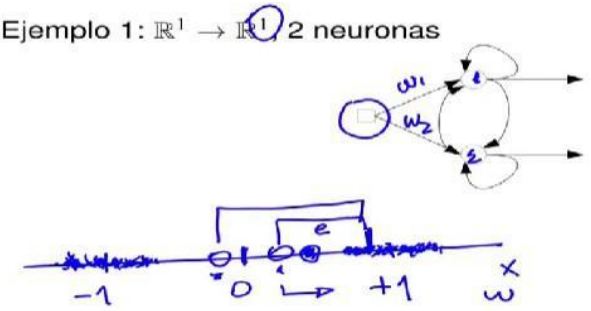
→ Se definen **3 etapas básicas en el entrenamiento**:

1. Ordenamiento global (o topológico)
   - $\Lambda_G(n)$ grande ($\approx$ medio mapa)
   - $\eta(n)$ grande (entre 0.9 y 0.7)
   - Duración 500 a 1000 épocas

2. Transición
   - $\Lambda_G(n)$ se reduce linealmente hasta 1
   - $\eta(n)$ se reduce lineal o exponencialmente hasta 0.1
   - Duración ≈1000 épocas
3. Ajuste fino (o convergencia)
   - $\Lambda_G(n) = 0$ (sólo se actualiza la ganadora)
   - $\eta(n) = cte$ (entre 0.1 y 0.01)
   - Duración: hasta convergencia (≈ 3000 épocas)

Formación de mapas topológicos

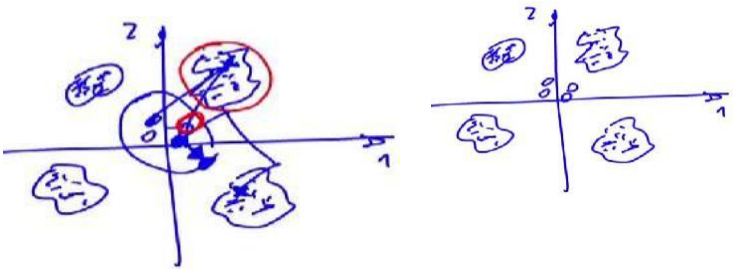
Ejemplo R1 → R1, **sin entorno/relación/vecindad** entre las neuronas.



→ la neurona 1 se va a ir corriendo hacia el +1 y la dos hacia el -1.

=> Aprende la estructura de los datos.

Ejemplo R2 → R1, **sin entorno/relación/vecindad** entre las neuronas. Distribución de los patrones: >
y pesos iniciales

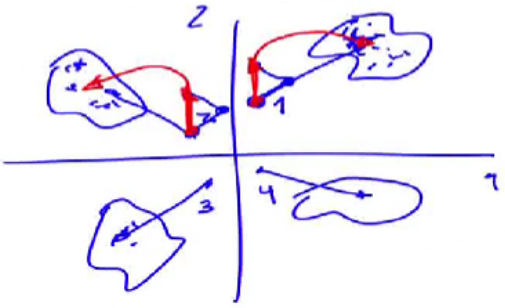


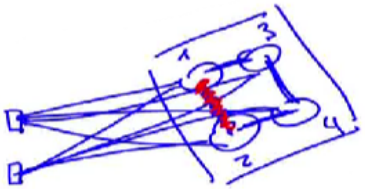
En su mayoría, cada iteración va a hacer que cada neurona se acerque a 1 grupo de patrones en particular, es por eso también que se hacen las 3 etapas básicas de entrenamiento.

=> Aprende la estructura de los datos.

→ **Ahora, sí sí consideramos una vecindad/entorno:**

- ➢ 2 o más neuronas se van a actualizar de forma conjunta, por ejemplo, van a subir juntas.
- ➢ Pero una vez que se inhiba el efecto del entorno, sí van a poder moverse más refinadamente por separado.



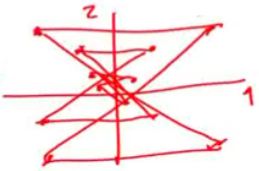


=> Las conexiones topológicas entre neuronas tienen una importancia, dado que **van a hacer que neuronas que están cercanas, representen regiones cercanas del espacio de solución.**

→ y es así cómo se genera el ordenamiento topológico. ← ← ← ← ← ← ← ← ←

Gráficamente: (2D)

→ **Podríamos visualizar el ordenamiento topológico** a lo largo del tiempo si graficamos como puntos a los pesos y trazamos líneas entre las neuronas que están conectadas entre sí.



=> Acá vemos el interés práctico de los SOM, que yo pueda proyectar un espacio de R^N, que no puedo visualizar, en un mapa bidimensional de neuronas en el que las neuronas que están cerca agrupan patrones que también están cerca en N dimensiones. ←←←

- Ideas de cómo entrenarlo:
  - algoritmo k-means etiquetado para clasificación
  - SOM etiquetado como clasificador...
  - algoritmo supervizado LVQ

→ Podríamos decir que los prototipos son los centroides que encontramos con k-medias.
→ Un SOM
→ o LVQ → Utiliza la información de clases durante el entrenamiento.

Algoritmo de aprendizaje LVQ1

1. Inicialización aleatoria
2. Selección:

$$c(n) = \arg\min_i \{\|\mathbf{x}(n) - \mathbf{m}_i(n)\|\}$$

3. Adaptación:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c,d,n)\alpha\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$s(c,d,n) = \begin{cases} +1 & \text{si } c(n) = d(n) \\ -1 & \text{si } c(n) \neq d(n) \end{cases}$$

4. Volver a 2 hasta satisfacer error de clasificación

El factor ***s(c,d,n)*** → nos dice si la neurona o prototipo ganador acertó a la clase o no.

→ Si acertó, acercamos el prototipo/neurona/vector de pesos al patrón que acaba de entrar.
→ Si no acertó, lo alejamos.

→ es casi igual que el SOM, pero:
- **No hay** un concepto de **vecindad**.
- Usamos el factor **s** => **es SUPERVISADO**.
- Cada prototipo se va a ir acercando al centro de la clase que representa más rápidamente, porque no solo acercamos, sino que cuando erra lo alejamos.

Observaciones:
★ No hay arquitectura neuronal
★ Se puede ver al cuantizador como **SOM lineal, sin entorno y supervisado**.
★ La velocidad de aprendizaje no puede ser exactamente la misma durante todo el entrenamiento →

LVQ1: velocidad de aprendizaje

Ecuación original:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

Si aplicamos distributiva del factor s(n) y la vel. de aprendizaje:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n) - s(n)\alpha(n)\mathbf{m}_c(n)$$

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)]\,\mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n)$$

Si expresamos al $\boldsymbol{m_c(n)}$ en función de cómo se calculó el anterior:

$$\mathbf{m}_c(n+1) =$$

$$= [1 - s(n)\alpha(n)] \{\mathbf{m}_c(n-1) + s(n-1)\alpha(n-1)\,[\mathbf{x}(n-1) - \mathbf{m}_c(n-1)]\} + s(n)\alpha(n)\mathbf{x}(n)$$

$\mathbf{x}(n-1)$ es afectado dos veces por α

Siendo $\alpha < 1$ la importancia relativa de los primeros patrones de entrenamiento siempre será menor que la de los últimos.

Si queremos que α afecte por igual a todos los patrones deberemos hacerlo decrecer con el tiempo.

=> hay que cambiarle el alfa de manera que todos los patrones pesen por igual en la posición final que tome el centroide.

=> se debe cumplir que el alfa en 2 iteraciones consecutivas pese lo mismo:

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)]\,\alpha_c(n-1)$$

$\alpha_c(n) = 1\,\alpha_c(n-1) - s(n)\,\alpha_c(n)\,\alpha_c(n-1)$

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$$

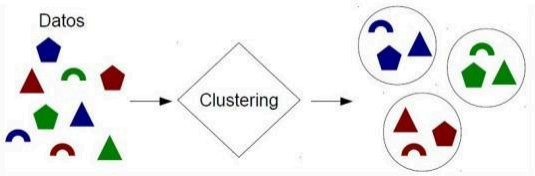
Aprendizaje no supervisado = Clustering (clase adicional)

Clustering

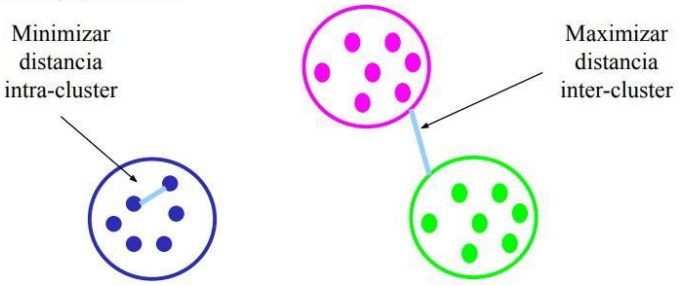
Agrupar objetos similares basados en alguna noción de similaridad. El resultado es una partición de los datos.

Cluster

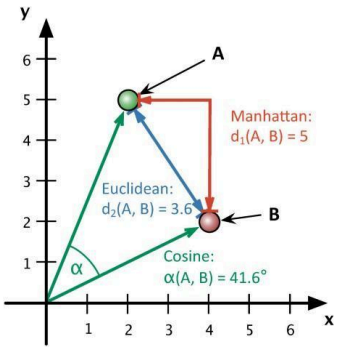
Un conjunto de objetos que son similares entre ellos, mientras que difieren de los objetos en otros clusters.



Objetivos del clustering:

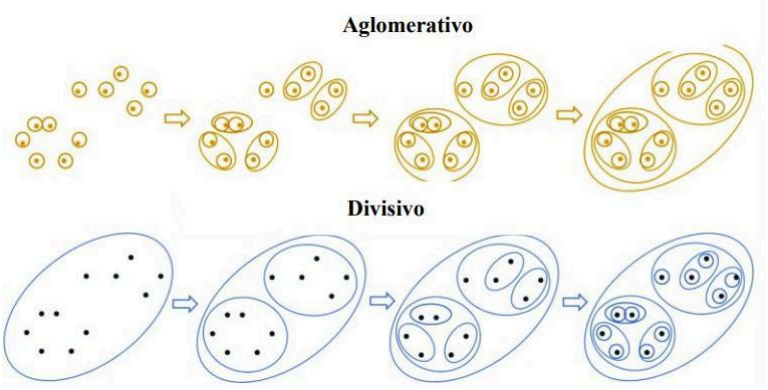


La distancia en el clustering



Clustering - Métodos

Agrupamiento jerárquico:

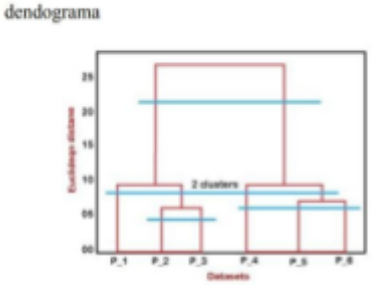


Para aplicarlo necesito una matriz de distancia:

- datos: P1, P2, P3, P4, P5 y P6
- calcular la matriz de distancia entre todos los datos

Divisivo:

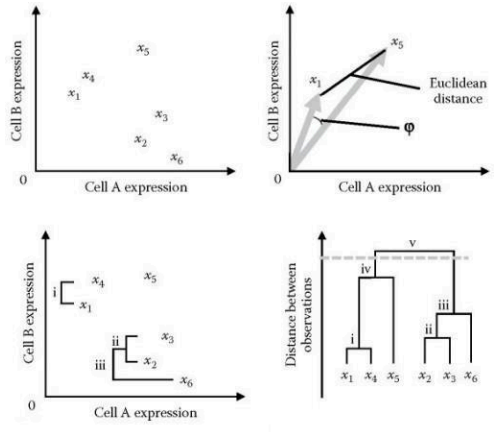
→ Con esto hacemos un:

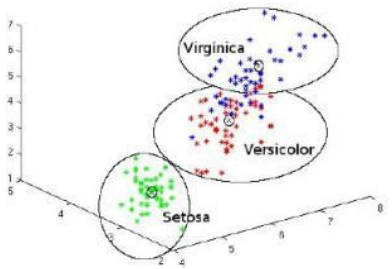


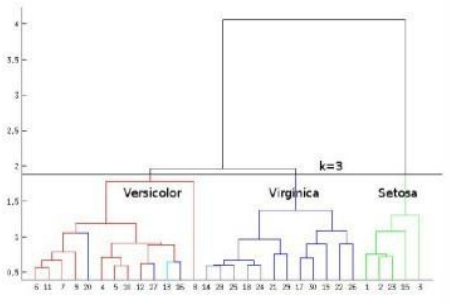
	P1	P2	P3	P4	P5	P6
P1	0.00					
P2	0.24	0.00				
P3	0.22	0.15	0.00			
P4	0.37	0.20	0.15	0.00		
P5	0.34	0.14	0.28	0.29	0.00	
P6	0.23	0.25	0.11	0.22	0.39	0.00

→ Unimos con una rama puntos que están cercanos con ramas, y trazamos luego líneas cuya altura determina la cantidad de clusters.

Aglomerativo:







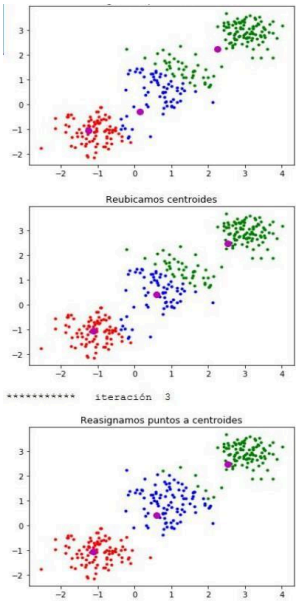
K medias:

- Algoritmo muy popular y muy simple.
- Particiona los datos en k clusters.
- Utiliza centroides (prototipos de los clusters).
- El problema es elegir el k.
- K-medias es un algoritmo que agrupa objetos en k grupos basándose en sus características.

- Minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster.
- Se suele usar la distancia euclídea.

El algoritmo consta de tres pasos:

1) **Inicialización:** una vez elegido el número de grupos k, se establecen k centroides en el espacio de los datos aleatoriamente.
2) **Asignación objetos a los centroides:** cada objeto de los datos es asignado a su centroide más cercano.
3) **Actualización de centroides**: se actualiza la posición del centroide de cada grupo tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo. Se repiten los pasos 2 y 3 hasta que los centroides no se mueven.



- Las principales ventajas del método k-medias son que es un método sencillo y rápido.
- Pero… es necesario decidir el valor de k a-priori
- El resultado final depende de la inicialización de los centroides.
- En principio no converge al mínimo global sino a un mínimo local.

Métricas de clustering

Cada algoritmo de clustering y combinación de parámetros produce particiones diferentes en los datos…

- Por lo tanto, ¿cómo evaluar los resultados obtenidos con varios algoritmos y varias configuraciones?
- ¿Cómo saber si la estructura encontrada por un algoritmo de clustering en los datos es válida?

Medidas internas:

Reciben como entradas los datos y las soluciones de clustering obtenidas, se evalúan según la información intrínseca de los datos

- **Cohesión**
  - Evalúa qué tan compactos u homogéneos son los clusters obtenidos por una solución. Se mide con la varianza intra-cluster:

$$\overline{C}_j = \frac{1}{|\Omega_j|} \sum_{\forall \mathbf{x}_i \in \Omega_j} \|\mathbf{x}_i - \mathbf{w}_j\|_2,$$

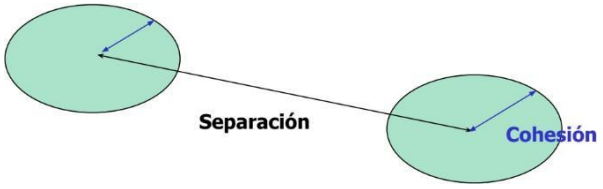
donde $|\Omega_j|$ es el número de datos en el cluster j, $\mathbf{w}_j$ es el centroide del cluster j, $\mathbf{x}_i$ es un dato agrupado en el cluster j.

  - Medida global de cohesión => promedio sobre todos los clusters-
  - Cohesión cercana a 0 => clusters más compactos (mejor)

- **Separación**
  - Se cuantifica el grado de separación entre k clusters individuales. Esta separación puede ser evaluada por la distancia euclídea media, mínima y máxima entre los centroides $\mathbf{w}_i$ y $\mathbf{w}_j$ de los clusters:

$$\begin{aligned} \overline{S} &= \frac{2}{k^2-k}\sum_{i=1}^{k}\sum_{j=i+1}^{k}\|\mathbf{w}_i-\mathbf{w}_j\|_2, \\ S_m &= \min_{0<i\neq j\leq k}\{\|\mathbf{w}_i-\mathbf{w}_j\|_2\}, \\ S_M &= \max_{0<i\neq j\leq k}\{\|\mathbf{w}_i-\mathbf{w}_j\|_2\}, \end{aligned}$$

  - Separación cercana a 0 => indica cercanía entre nodos.
  - Separación cercana a 1=> clusters más separados (mejor)



- **Davies Bouldin**
  - Medida combinada, función de la relación entre la suma de las dispersiones
  - intra-nodo y la separación entre nodos. <u>Índice de Davies-Bouldin:</u>

$$DB = \frac{1}{k}\sum_{i=1}^{k}\max_{j\neq i}\left(\frac{\overline{C}_i+\overline{C}_j}{\|\mathbf{w}_i-\mathbf{w}_j\|_2}\right).$$

cohesión media

separación media

  - Medida del solapamiento entre nodos → privilegia agrupamientos compactos y bien separados entre sí.
  - Valores de DB cercanos a 0 indican poco solapamiento

Medidas externas:

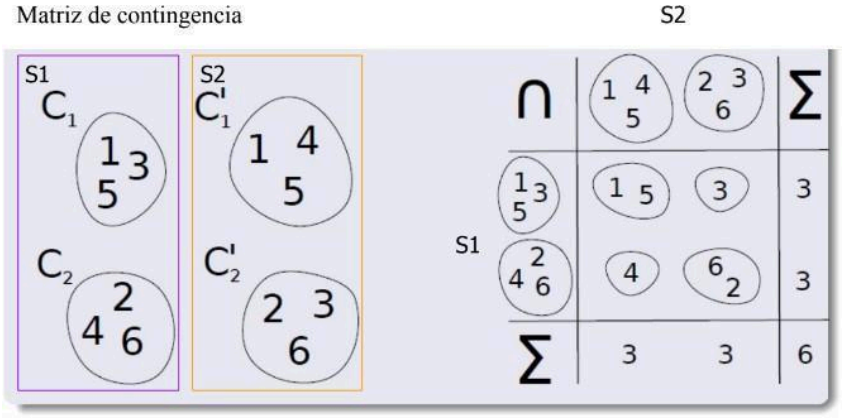
<u>Comparan la similitud existente entre un par de soluciones de clustering (**S1** y **S2**), o contra una referencia (“gold standard”)</u>

- **FM - Fowlkes and Mallows**
  - Se utiliza para determinar la similitud entre dos particiones de clusterings (S1 y S2).

$$\mathrm{FMI} = \frac{\mathrm{TP}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})}}$$

    - **TP**: nro. de pares de puntos que están presentes en el mismo cluster en S1 y en S2
    - **FP**: nro. de pares de puntos que están presentes en el mismo cluster en S1 pero NO en S2
    - **FN**: nro. de pares de puntos que están presentes en el mismo cluster en S2 pero NO en S1

- Puede ser entre un agrupamiento y una clasificación de referencia.
- Un valor más alto (cercano a 1) para el índice de FM indica una mayor similitud entre las soluciones.



- **Rand Index (RI)**
  - Para 2 soluciones S1 y S2, medimos:
    a) nro. de pares de elementos que están en el mismo conjunto en S1 y S2
    b) nro. de pares de elementos que están en conjuntos diferentes en S1 y en conjuntos diferentes en S2
  - El índice de Rand es: $$\mathrm{RI} = \frac{a+b}{C_2^{n_{samples}}}$$

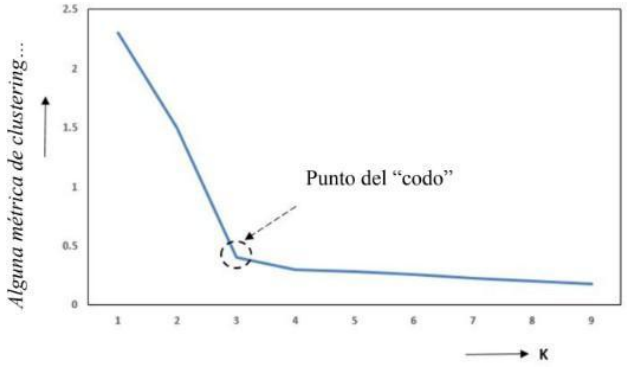
    donde el denominador es el número total de pares posibles en el conjunto de datos.
  - El etiquetado perfecto tiene una puntuación de 1.

- **Jaccard, NMI, …**

El número de clusters (k) - K óptimo

Las métricas de clustering pueden ser usadas para *estimar el número adecuado de clusters para un dataset.*

- Eso involucra **calcular los resultados de una métrica de clustering para un rango de valores de k, y plotear la curva resultante**.
- El mejor nro. de clusters puede determinarse como un **“pico” o “codo” en la curva** de performance obtenida.



Programación diferenciable - ¿Qué pasa si conectamos muchas neuronas?

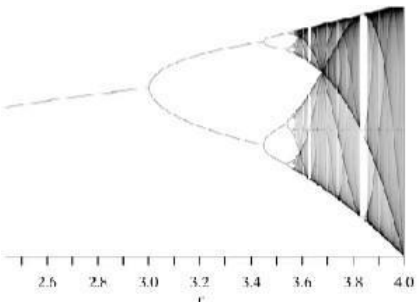
- Necesidad de contar con grandes cantidades de datos para entrenar
  → Revolución de los grandes datos
- Alto costo computacional
  → Unidades de procesamiento gráfico GPUs
- Desvanecimiento del gradiente
  → Pre-entrenamiento no-supervisado,
  → Auto-codificadores con máquinas de Boltzmann,
  → Normalización por lotes,
  → Redes residuales,
  → Otros condimentos: ReLU, dropout, pooling..

- Dificultad para construir y entrenar modelos muy grandes y con diferentes tipos de capas neuronales interconectadas
  → **Programación diferenciable ∂P**

Programación diferenciable ∂P

Es una mezcla entre la diferenciación numérica y la simbólica.

→ Supongamos que tenemos que calcular las derivadas del mapa logístico: $\ell_{n+1} = r\ell_n(1-\ell_n)$, con respecto a x, a lo largo de las iteraciones n.



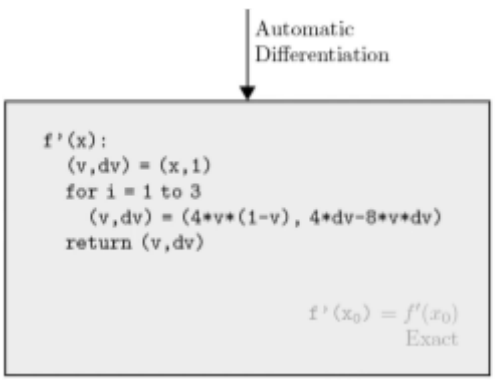
Con $r = 4$, por ejemplo, tenemos

$\ell_{n+1} = 4\ell_n(1-\ell_n), \ell_1 = x$

1	x	1	1
2	$4x(1-x)$	$4(1-x)-4x$	$4-8x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2-16x(1-2x)^2-64x(1-x)(1-2x)$	$16(1-10x+24x^2-16x^3)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2)+64(1-x)(1-2x)^2(1-8x+8x^2)^2-64x(1-2x)^2(1-8x+8x^2)^2-256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1-42x+504x^2-2640x^3+7040x^4-9984x^5+7168x^6-2048x^7)$

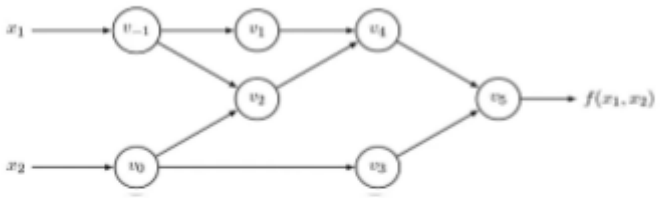
Derivación automática:

- La idea es ir acumulando en un bucle las derivadas de la función, guardo un par de variables, el valor de la función, y el valor de la derivada de la función.
- De esta manera calculamos exactamente la derivada con una mezcla de cómputo simbólico con acumulación de información numérica.



–> MODO HACIA ADELANTE

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

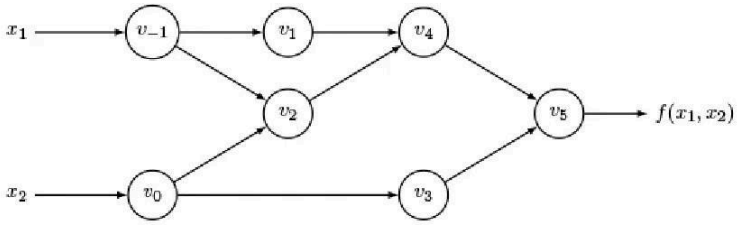


$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

v_{-1}	$= x_1$	$= 2$	$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
v_0	$= x_2$	$= 5$	$\dot{v}_0$	$= \dot{x}_2$	$= 0$
v_1	$= \ln v_{-1}$	$= \ln 2$	$\dot{v}_1$	$= \dot{v}_{-1}/v_{-1}$	$= 1/2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$	$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
v_3	$= \sin v_0$	$= \sin 5$	$\dot{v}_3$	$= \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$	$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$	$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
y	$= v_5$	$= 11.652$	$\dot{y}$	$= \dot{v}_5$	$= 5.5$

–> MODO REVERSO

Hago el de todas las entradas en una sola pasada, con solo saber las derivadas locales.



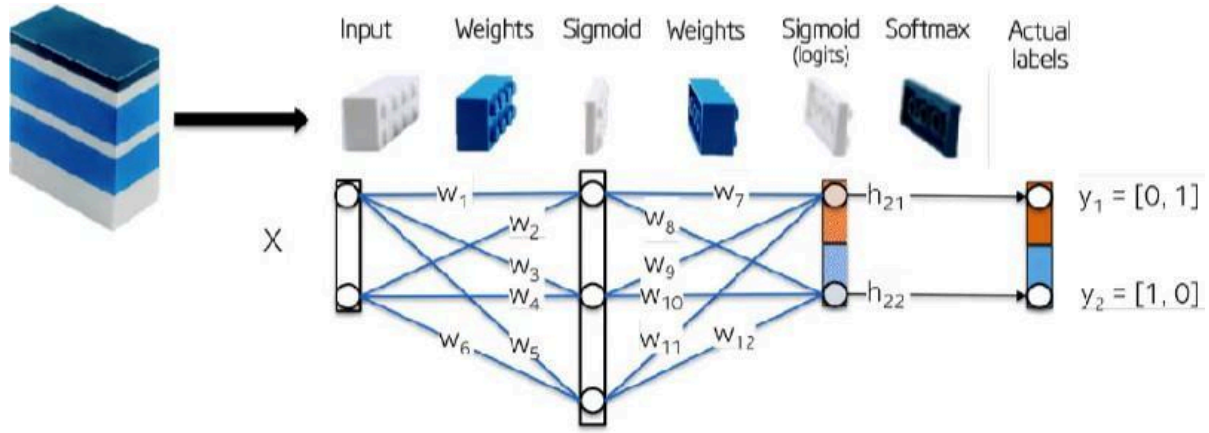
$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

v_{-1}	$= x_1$	$= 2$	$\bar{x}_1$	$= \bar{v}_{-1}$		$= 5.5$
v_0	$= x_2$	$= 5$	$\bar{x}_2$	$= \bar{v}_0$		$= 1.716$
v_1	$= \ln v_{-1}$	$= \ln 2$	$\bar{v}_{-1}$	$= \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}}$	$= \bar{v}_{-1} + \bar{v}_1 / v_{-1}$	$= 5.5$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$	$\bar{v}_0$	$= \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0}$	$= \bar{v}_0 + \bar{v}_2 \times v_{-1}$	$= 1.716$
			$\bar{v}_{-1}$	$= \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}}$	$= \bar{v}_2 \times v_0$	$= 5$
v_3	$= \sin v_0$	$= \sin 5$	$\bar{v}_0$	$= \bar{v}_3 \frac{\partial v_3}{\partial v_0}$	$= \bar{v}_3 \times \cos v_0$	$= -0.284$
v_4	$= v_1 + v_2$	$= 0.693 + 10$	$\bar{v}_2$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_2}$	$= \bar{v}_4 \times 1$	$= 1$
			$\bar{v}_1$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_1}$	$= \bar{v}_4 \times 1$	$= 1$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$	$\bar{v}_3$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_3}$	$= \bar{v}_5 \times (-1)$	$= -1$
			$\bar{v}_4$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_4}$	$= \bar{v}_5 \times 1$	$= 1$
y	$= v_5$	$= 11.652$	$\bar{v}_5$	$= \bar{y}$	$= 1$	

→ Nos permite no tener que sacar la derivada de la ecuación completa, solo conociendo las funciones y sus dd en cada “neurona”

→ Soluciona el desvanecimiento del gradiente (son exactas).

Programación diferenciable - el Lego de las arq. neuronales profundas



→ Permite poder manipular elementos de nuestro modelo sin tener que preocuparnos por volver a hacer cálculos engorrosos.

→ Permitió experimentar mucho más con las redes neuronales.

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
  - Perceptrones simples y de una capa
  - Perceptrón multicapa
  - Redes con funciones de base radial
  - ...
- Redes recurrentes (recurrent/with feedback)
  - Redes de Kohonen
  - Redes de Hopfield
  - Redes de Boltzmann
  - Redes de Elman/Jordan
  - Modelos de resonancia adaptativa
  - ...
- Otros modelos híbridos
  - Parcialmente recurrentes
  - Parcialmente conectados
  - Redes modulares
  - ...

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
  - Deep Belief Network (DBN)
  - Convolutional Neural Network (CNN)
  - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
  - ...
- Redes recurrentes (recurrent/with feedback)
  - Long Short-Term Memory (LSTM)
  - Attention Mechanisms
  - Transformers
  - ...
- Otros modelos y métodos de entrenamiento
  - Generative Adversarial Networks (GAN)
  - Deep Reinforcement Learning (DRL)
  - Bayesian Deep Learning (BDL)
  - ...

Bases estadísticas del reconocimiento de patrones

Aprendizaje Maquinal

Problema: ¿Cómo enseñar a una máquina a que emule las tareas humanas de detección, percepción, identificación, reconocimiento, etc.?

Objetivo: Programar máquinas para:
- ➢ aprender a realizar una *tarea*,
- ➢ mejorando su *desempeño*
- ➢ basado en la *experiencia.*

→ **Restricciones adicionales**:
- mínima intervención humana posible,
- con reducido conjunto de ejemplos,
- etc…

Conceptos básicos de Aprendizaje Maquinal

Podemos considerar dos ramas de la Inteligencia Artificial (IA):

- **IA clásica:**
  - Modelado del proceso de *razonamiento* humano mediante técnicas de la Lógica.
  - Aprendizaje deductivo.

- **Reconocimiento de Formas:**
  - Modelado del proceso de *percepción* humano mediante técnicas de la *Teoría de la Decisión Estadística* y de la *Teoría de los Lenguajes Formales.*
  - Aprendizaje inductivo. → A través de ejemplos

Vista como una disciplina de la IA… Aspectos de Aprendizaje Maquinal:

- ***Adquisición y representación del conocimiento***: conformación y almacenamiento de conjuntos de patrones o prototipos.
- ***Aprendizaje***: algoritmos de aprendizaje inductivo a partir de un conjunto de entrenamiento.
- ***Clasificación*****:** etiquetado de patrones nuevos utilizando el conjunto de clases disponible.
- ***Evaluación***: mecanismos para evaluar la bondad, confianza o error del sistema.

Definiciones

★ **Patrón:**
- ➢ *Objeto de interés que es identificable del resto.*
- ➢ Posiblemente difusos, no bien definidos, no visibles o tangibles.
- ➢ Ej: una huella digital, la voz de una persona, una cara, etc.

★ **Reconocimiento de patrones (RP):**
- ➢ Estudio de los procesos de percepción y razonamiento humanos:
  - Con estas 3 subtareas:

- *capacidad de distinguir y aislar los patrones,*
- *reunirlos en grupos,*
- *asignarles un nombre identificatorio a cada grupo.*

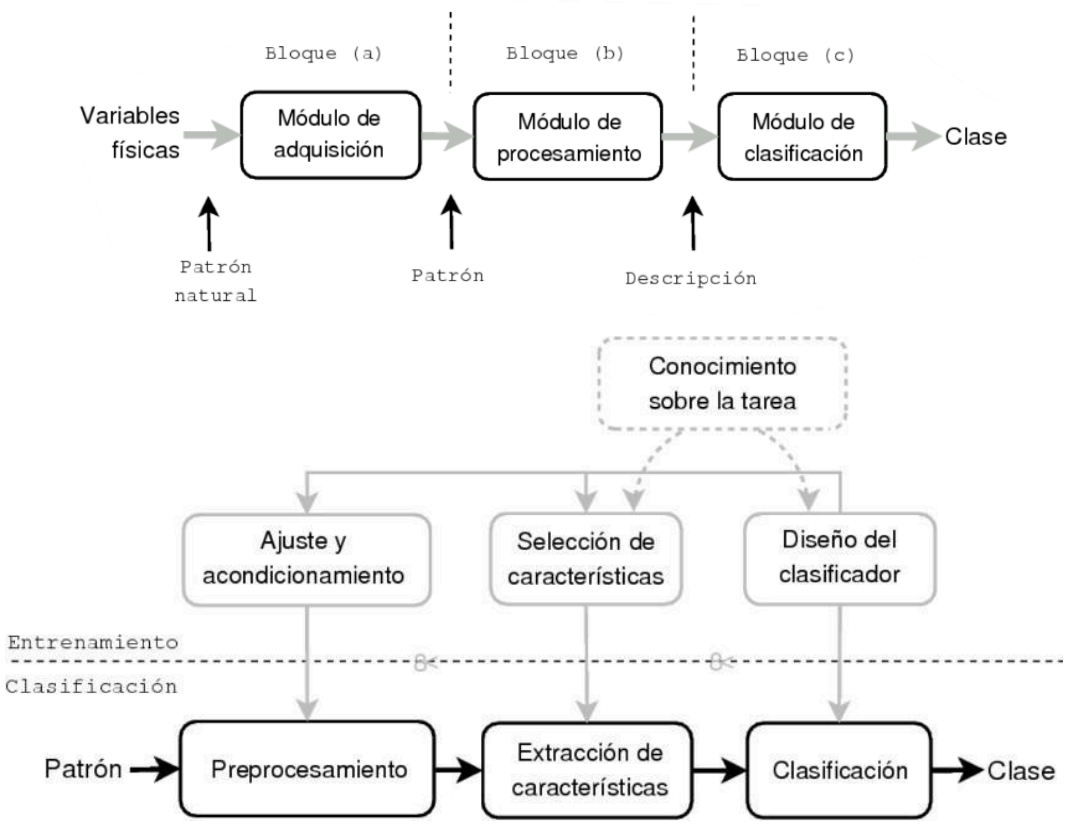
➢ Objetivo del RP: crear sistemas informáticos que imiten el comportamiento descrito.

Paradigma de trabajo conceptual - Diagrama en bloques

(a) **Adquisición**: transducción del mundo real a la representación digital.

(b) **Procesamiento digital**: acondicionamiento y representación alternativa, permite caracterizar los objetos. Como salida tiene la descripción, que es un vector, tiene que ser pequeño y lo más representativo posible.

(c) **Clasificación**: decisión sobre la clase.



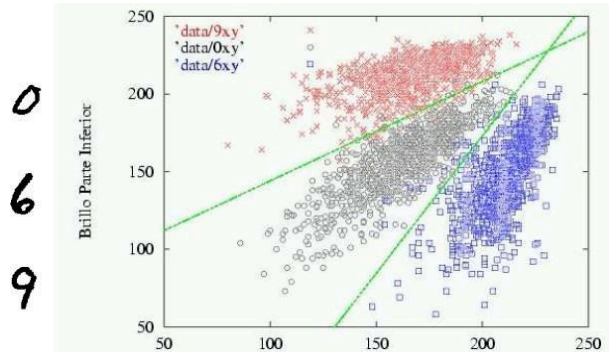
Aproximaciones

- Aproximación geométrica o estadística:
  - Basada en la Teoría Estadística de la Decisión.
  - Representación de *patrones* como ***vectores numéricos.***
  - Representación de *clases* mediante ***patrones prototipo***.
  - Tipos de clasificadores: gaussianos, basados en distancia, etc.

- Aproximación estructural o sintáctica:
  - Basada en la Teoría de Lenguajes Formales.
  - Representación de *patrones* como ***cadenas de símbolos***.

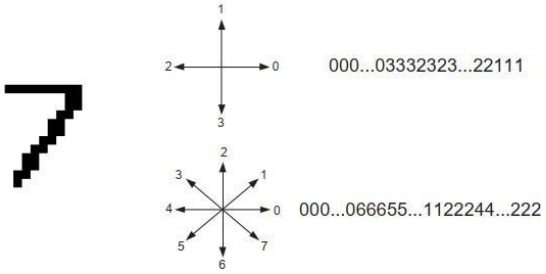
- Utilización de *reglas sintácticas* para especificar los ***patrones válidos de una clase.***
- Tipos de clasificadores: autómatas, gramáticas, HMM, etc

Ejemplo de aproximación geométrica:

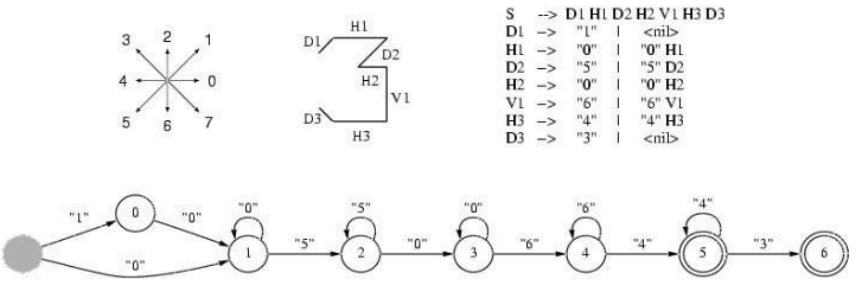


Ejemplo de aproximación sintáctica:

Descripción de dígitos mediante cadenas de contorno:



Clasificador sintáctico basado en autómatas:



Conceptos de <u>clasificación estadística</u>

- ***Adquisición y preproceso de datos***: vector numérico que representa al patrón natural. Formalmente, un patrón es una variable aleatoria *n*-dimensional $y = [y1\ y2\ \ldots\ yn]^T$, con
  $y_i \in R$ para $i = 1, 2, \ldots, n$, que representa un punto en el espacio de patrones $P \in R^n$.

- ***Extracción de características:*** información *relevante* para la clasificación.

  Formalmente, dado un conjunto de patrones *n*-dimensionales y se trata de obtener un <u>nuevo conjunto de representaciones</u>

  $$x = [x_1\ x_2\ \ldots\ x_d]^T \text{ , donde } \boldsymbol{d \leq n.}$$

  - ➢ Cambio en el espacio de representación: transformaciones lineales que maximizan la varianza *(d = n)*.
  - ➢ Reducción de dimensionalidad de los datos *(d < n)*.

- ***Espacio de características:*** $E \in R^d$.

- ***Extracción de características:*** *propiedades deseables*
  - ➢ <u>Precisión</u>: representaciones diferentes para objetos diferentes.
  - ➢ <u>Unicidad o determinismo</u>: representación única para cada objeto.

- ➢ Continuidad en el espacio: da robustez al sistema, significa que en un punto en ese espacio que tiene una etiqueta, con cualquier modificación de ruido, el sistema siga identificando esa etiqueta. => Inmunidad al ruido y capacidad de generalización.

(las primeras 2 se pueden relajar)

- ***Clases informacionales:*** *salidas del sistema*
  - ➢ Número de clases: *C*
  - ➢ Conjunto de (etiquetas de) clases: Ω = {ω1, ω2, . . . , ωc}
  - ➢ Conjunto extendido: Ω∗ = {ω1, ω2, . . . , ωc, ω0}, donde ω0 es la *clase de rechazo* → Casos donde el sistema no tiene seguridad sobre la desición que tomó (indica revisar la muestra).

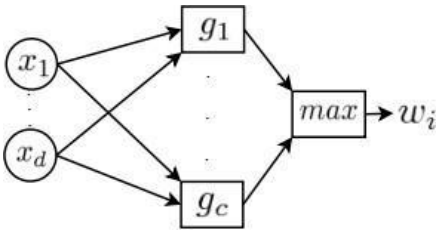
Definiciones de clasificación estadística

Clasificador estadístico: máquina formada por c (funciones) discriminantes

$g_i : E \rightarrow R,\ 1 \leq i \leq c$ → tenemos tantas funciones como clases.

tal que dado un patrón $\boldsymbol{x} \in E$,

x se asigna a la clase ω_i si $g_i(x) > g_j(x)\ \forall j \neq i$



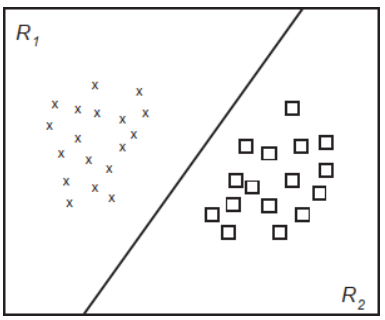
Regiones y fronteras de decisión

- Regiones de decisión: un clasificador divide el espacio en ***c*** regiones de decisión **R1, R2, . . . , Rc**, tal que

$$R_i = \{x \in E : g_i(x) > g_j(x)\ \forall j \neq i\}$$

- Fronteras de decisión: superficies del espacio que separan regiones de decisión contiguas.
  - Son los puntos del espacio donde hay igualdad en la salida de las funciones discriminantes → Causantes de rechazo.

Hipersuperficies definidas por: $g_i(x) - g_j(x) = 0,\ i \neq j,\ 1 \leq i, j \leq c$



Clasificadores estadísticos básicos

Las FD son combinaciones lineales o cuadráticas de las componentes del vector de características.

- **Clasificador lineal:**

$$g(\mathbf{x}) = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^t \mathbf{x} + w_0$$

→ Perceptrón multicapa

Fronteras: hiperplanos.

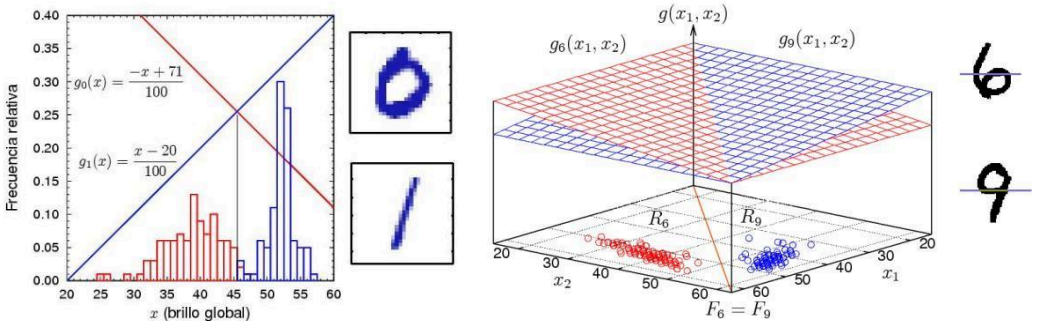
- **Clasificador cuadrático:**

$$g(\mathbf{x}) = \sum_{i=1}^{d} \sum_{j=1}^{d} w_{ij} x_i x_j + \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{x}^t W \mathbf{x} + \mathbf{w}^t \mathbf{x} + w_0$$

Fronteras: hipercuádricas.

Ejemplo: OCR de dígitos manuscritos

- Unidimensional: brillo global.
- Bidimensional: brillo de mitad superior (x_1) e inferior (x_2).



Definiciones de teoría de la decisión

- **Probabilidad a priori P(ωi)** de una clase ωi: probabilidad de que una muestra arbitraria pertenezca a ωi.

> "Probabilidad de observar la etiqueta c sin saber qué muestra es"

Puede verse como <u>la **proporción** de muestras de ωi respecto al total</u> (conocimiento que se tiene antes de hacer los experimentos).

Condiciones:
- $0 \leq P(\omega_i) \leq 1$, para $i = 1, \ldots, c$.
- $\sum_{i=1}^{c} P(\omega_i) = 1$.

Clasificador trivial de dos clases basado en probabilidades a priori:
- Decidir por ω_1 si $P(\omega_1) > P(\omega_2)$
- Decidir por ω_2 si $P(\omega_1) < P(\omega_2)$

- **Densidad condicional P(x|ωi)** de una clase ωi: función de densidad de probabilidad que caracteriza la distribución estadística de las muestras de ωi.

> "Probabilidad de observar la muestra x sabiendo que la etiqueta es c"

El vector de características x se considera una variable aleatoria d-dimensional de función de densidad **P(x|ωᵢ)** cuando las muestras pertenecen a **ωᵢ**.

Condiciones:

- $P(\mathbf{x}|\omega_i) \geq 0$, para $i = 1, \ldots, c$.
- $\int_E P(\mathbf{x}|\omega_i)\ d\mathbf{x} = 1$.

- **Probabilidad conjunta P (x, ωi)** muestra-clase se define como:

$$P(\mathbf{x}, \omega_i) = P(\omega_i)P(\mathbf{x}|\omega_i)$$

"Probabilidad de observar la muestra x con la etiqueta c"

- **Densidad incondicional P(x)** de las muestras se define como

$$P(\mathbf{x}) = \sum_{j=1}^{c} P(\mathbf{x}, \omega_j) = \sum_{j=1}^{c} P(\mathbf{x}|\omega_j)P(\omega_j)$$

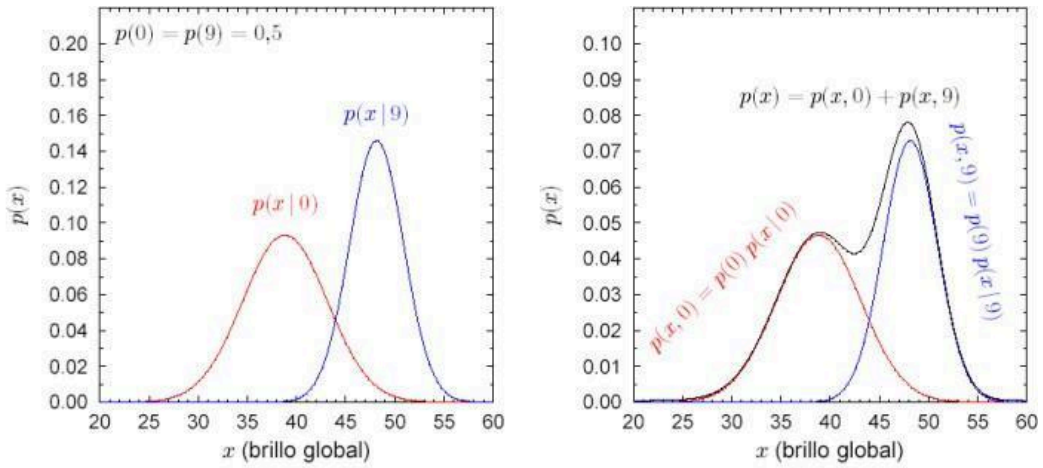
La suma para todas las clases de la probabilidad conjunta

"Probabilidad de observar la muestra x sin saber cuál es su etiqueta"

→ Caracteriza la distribución estadística de las muestras con independencia de las clases a las que pertenecen.

Se cumple que: $P(\mathbf{x}) \geq 0$, y $\int_E P(\mathbf{x})\ d\mathbf{x} = 1$.

Ejemplo:



Significado de: $P(\omega = 0) = 0{,}5;\ P(x = 45|\omega = 0) = 0{,}033;$
$P(x = 45, \omega = 0) = 0{,}016;\ P(x = 45) = 0{,}054.$

Supongamos que tenemos conocidas **P(ωi)**, P(x|ωi) y una medida de **x**. **¿Cómo influye este conocimiento sobre la decisión del clasificador?**

Probabilidad a posteriori P(ωi|x) de una clase ωi: ***probabilidad de que una muestra particular x pertenezca a ωi.***

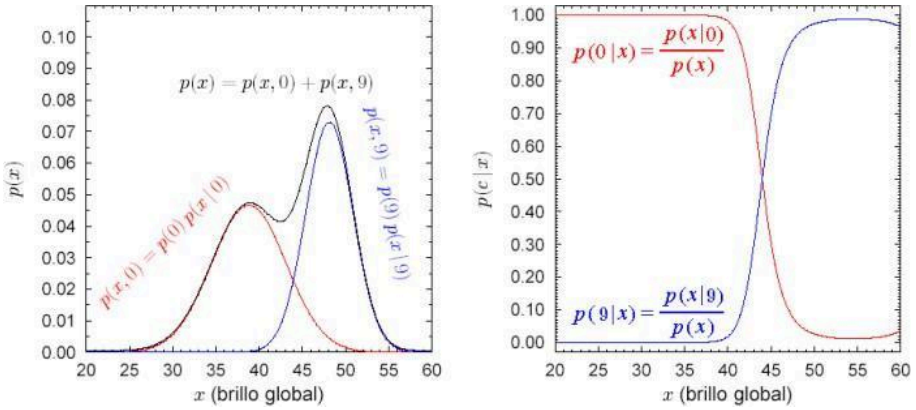
"Probabilidad de observar la etiqueta c sabiendo que la muestra es x"

Se calcula mediante la **regla de Bayes:**

$$P(\omega_i|\mathbf{x}) = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{P(\mathbf{x})} = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{j=1}^{c} P(\mathbf{x}|\omega_j)P(\omega_j)}$$

Se cumple que: $0 \leq P(\omega_i|\mathbf{x}) \leq 1$, y $\sum_{i=1}^{c} P(\omega_i|\mathbf{x}) = 1$.

→ Puede verse como una actualización de P (ωi) después de observar x.



=> Estas probabilidades serían las funciones discriminantes que estamos buscando.

Clasificador de Bayes

- **Regla de clasificación de Bayes:** asignar a x la clase con mayor probabilidad a posteriori.

$$\hat{\omega} = \quad \arg\max_{\omega_i : 1 \leq i \leq c} \quad P(\omega_i|\mathbf{x})$$

- **Clasificador de Bayes:**
  - Utiliza las probabilidades a posteriori como funciones discriminantes.
  - Es de mínimo error basado en FD monótonas crecientes(.)

$$\begin{aligned} g_i(\mathbf{x}) &= P(\omega_i|\mathbf{x}) = \frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{P(\mathbf{x})} \\ &\equiv P(\mathbf{x}|\omega_i)P(\omega_i) \\ &\equiv \log P(\mathbf{x}|\omega_i) + \log P(\omega_i) \end{aligned}$$

Equivalencia entre clasificadores:

$$(g_1, \ldots, g_c) \equiv (f(g_1), \ldots, f(g_c)) \equiv (g'_1, \ldots, g'_c) \text{ si } R_c = R'_c \ \forall c.$$

- Error de Bayes a posteriori: probabilidad de error puntual dado por

$$P(\text{error}|\mathbf{x}) = 1 - \max_{1 \leq i \leq c} P(\omega_i|\mathbf{x})$$

- Probabilidad media de error del clasificador:

$$P(\text{error}) = \int_E P(\text{error}|\mathbf{x})P(\mathbf{x})d\mathbf{x}$$

→ **Problemas con el cálculo analítico del error**:

- **P(ωi)** y **P(x|ωi)** desconocidas.
- Regiones de integración complejas.

→ **Solución**: *estimación* del error.

Aplicaciones del RP

Reconocimiento de Imágenes

OCR - Análisis de documentos - Rec. de firmas - Identif. de patentes - Detección de defectos en piezas industriales (control de calidad) - Rec. de gestos faciales

Reconocimiento de Habla y Lenguaje

Rec. de palabras aisladas y discurso continuo - Identif. del locutor - Traducción automática - Comprensión

Aplicaciones Biomédicas

Segmentación de tejido en imágenes médicas - Bio Identificación (huellas dactilares, palma, iris, otros)

Economía

Minería de datos - Detección de patrones de fraude

Aplicaciones en astronomía, agricultura, protección civil, etc.
